

HybriChain 毕业答辩演讲稿

预估时长：8-10 分钟（中等语速约 200 字/分钟）

[第 1 页 标题页]

各位老师好，我是四川大学网络空间安全学院的学生，本科毕业论文题目是"HybriChain：基于 eBPF 的开源 AI 推理引擎的跨语言调用链追踪方法"。下面开始汇报。

[第 2 页 汇报结构]

本次汇报分七个部分：研究背景与动机、相关工作及其不足、整体方法与系统设计、Ollama 特殊工程问题及解决方案、实验设计与结果分析、本文创新点，以及总结与展望。

[第 3 页 研究背景]

当前，Ollama、llama.cpp 等开源推理框架已成为企业和个人部署大语言模型的主流选择。为了追求极致性能，底层架构普遍采用多语言混合编程与多进程解耦。

但与此同时，安全脆弱性也在不断暴露——比如 TorchServe 的 API 未授权上传导致远程代码执行，Ollama 的畸形 API 数据可造成目录遍历越权读取。核心矛盾在于：推理引擎内部的数据流转对安全运维人员几乎完全不透明，现有观测工具无法连贯还原从 API 接收到内核调用的完整调用链。

[第 4 页 相关工作及其不足]

我们从三个维度分析了现有方案。

第一是推理引擎本身。多语言混合编程加多进程解耦架构导致内部数据流极度复杂，且缺乏内建的可观测性接口，调用链追踪依赖外部工具。

第二是 eBPF 动态追踪技术。eBPF 已在内核级零侵入观测方面成熟应用，支持 uprobe 和 tracepoint 多锚点挂载，但现有应用多集中在网络和存储领域，尚未针对多语言 AI 推理引擎形成系统性的追踪方案。

第三是现有调用链追踪工具。strace 和 ftrace 只能观测系统调用层，无法深入用户态函数级行为；动态二进制插桩工具如 Valgrind、DynamoRIO 性能退化高达 10 到 100 倍，不可用于生产环境；APM 工具如 Jaeger、SkyWalking 需要代码侵入插桩，对跨语言边界支持薄弱。

它们的共性不足是：控制流在跨语言边界时完全断裂，各语言符号空间割裂，无法建立统一语义关联。

[第 5 页 研究目标与方法概述]

基于上述问题，我们设计了一套动静结合、低开销、通用的底层追踪框架，适用于所有采用多语言混合架构的 AI 推理引擎，具体包含四点目标：

第一，跨语言符号映射——将来自不同语言命名空间的函数符号统一映射到同一语义空间。

第二，eBPF 内核级追踪——利用 eBPF 在零侵入前提下，对推理引擎进行函数与系统调用的双层观测。

第三，统一调用链缝合——将用户态与内核态事件按时间戳缝合为端到端推理调用链，并赋予置信状态。

第四，调用链下游分析——基于端到端调用链图谱，支持异常行为检测、性能瓶颈定位、安全审计与容量规划等多种分析场景。

需要说明的是，本方法不依赖特定引擎实现，本论文以 Ollama 为实验靶点验证有效性，可推广至 llama.cpp、vLLM 等多语言推理框架。

[第 6 页 系统总体架构]

这是 HybriChain 的总体架构图。系统由三大核心模块构成：

静态符号提取模块负责离线解析推理引擎的二进制文件，提取各层函数符号；eBPF 内核追踪模块实时捕获系统调用与用户态函数调用事件流；缝合与分析引擎将静态拓扑与动态轨迹融合，输出带置信状态的调用链图谱。

[第 7 页 eBPF 探针下沉策略]

推理引擎每秒产生数万次调用事件，全量追踪将导致存储和计算资源迅速耗尽。HybriChain 采用探针下沉策略，将追踪点精确挂载至三个关键层次：

第一，推理计算函数层，在张量计算核心函数上挂载 uprobe，获取微秒级耗时数据。第二，系统调用层，追踪 read、write、futex、mmap、clone3 等与推理语义相关的系统调用。第三，内存分配层，专门追踪大块内存分配，分析 KV Cache 缓存行为。

eBPF 相比 strace 和 DTrace 的核心优势在于：内核运行、零侵入、热更新，以及 RingBuf 零拷贝传输。

[第 8 页 漏斗式降噪流水线]

原始内核事件流每秒数万条，其中大量是与推理无关的噪声。我们设计了五步漏斗降噪：

第一步是元信息过滤，去除追踪工具自身的启动和停止提示行。第二步是进程过滤，仅保留推理引擎相关进程的事件。第三步是类型白名单过滤，保留 read、write、futex、mmap 等核心调用，过滤 brk、mprotect、madvise 等无关系统调用。第四步是参数过滤，对 read 仅记录有效文件描述符，对 mmap 仅记录大于等于 1MB 的大块分配。第五步是栈帧清洗，过滤纯内存地址帧，保留可读符号名并进行 demangle 还原。

[第 9 页 置信度缝合算法]

降噪后的数据来自 uprobe 和 tracepoint 两类分散事件，处于不同时间精度——分别是微秒级和毫秒级，无法直接拼接。我们提出了基于时间戳对齐的启发式缝合算法，为每条跨语言边赋予三种置信状态：

Confirmed，即确认边，表示两端节点在两棵以上不同请求树中共同出现，可信度最高。Inferred，即推断边，表示仅在一棵请求树中共同出现，可能存在时间窗口误匹配。Unknown，即未知边，表示静态存在但动态未观测到，调用路径未被触发。

缝合输出为按时间排序的统一调用序列，每条事件携带置信状态与风险等级，便于下游分析。

[第 10 页 五层符号分类体系]

缝合后的调用链中每条边对应一个函数符号，但这些符号来自完全不同的命名空间。我们提出了五层语义分类模型，将所有函数符号统一归入以下层次：

f0 是系统边界层，对应 POSIX 标准函数如 open、read、write、mmap，与编程语言无关。f1 是运行时胶水层，对应语言绑定函数如 cgo 桥接、Python C Extension，是跨语言调用的关键节点。f2 是张量计算层，对应矩阵乘法、注意力机制等核心计算函数。f3 是内存管理层，对应缓存分配、模型权重加载、KV Cache 管理。f4 是调度协调层，对应任务分发、线程池管理、futex 同步操作。

这个分类体系使得来自不同语言层的符号可以在同一坐标体系下进行跨层调用分析，也为后续的异常检测提供了层次语义基础。

[第 11 页 Ollama 特殊工程问题及解决方案]

以 Ollama 为实验靶点验证方法有效性，需要解决多语言混合架构带来的两个特殊工程问题。

第一个是语言绑定函数命名不确定的问题。多语言混合架构中，语言绑定层编译器生成的函数符号含哈希后缀，无法直接 demangle，也无法进行标准化分类。我们的解决方案是通过识别语言绑定函数的前缀特征，提取原始函数名，将其正确分类至胶水层。该技术不绑定特定引擎，适用于所有多语言混合推理框架。

第二个是多进程通信隔离的问题。推理引擎的 daemon 进程与计算 runner 进程通过 Unix Domain Socket 通信，控制流跨越进程边界，单进程追踪无法缝合。我们的解决方案是在双进程上同时挂载 uprobe，通过请求 ID 关联跨进程事件，建立完整调用链。

[第 12 页 实验设计]

实验在 CPU-only 环境下进行，测试模型为 TinyLlama-1.1B 和 Qwen2-0.5B。

评估从三个维度展开：第一是调用链缝合准确性，基于 20 条 Ground Truth 基准路径验证；第二是系统性能开销，评估 eBPF 追踪对推理时延和内存的影响；第三是异常行为检测能力，验证调用链图谱在安全审计中的实际效果。对照组为无 eBPF 基线和 strace 全量追踪。

[第 13 页 实验结果：缝合准确性]

基于 20 条基准路径的验证结果显示：精确率达到全部确认，零假阳性，每一条确认边均真实可靠。

召回率方面，直接观测确认了 3 条边，占比 15%，含推断召回率为 12 条，占比 60%。召回率较低的原因是探针仅挂载了 4 个核心函数，大量子函数未被覆盖。值得强调的是，这 3 条确认边恰好覆盖了最有价值的跨语言边界，是调用链追踪最核心的部分。

[第 14 页 实验结果：性能开销]

性能开销方面，eBPF 追踪对推理时延的影响可忽略不计：TinyLlama 从 65ms 到 66ms，仅增长 1.5%；Qwen2 从 64ms 到 68ms，增长 6.3%。

在降噪压缩效果上，TinyLlama 从 25,329 条事件压缩至 8,891 条，保留 35%；Qwen2 从 13,670 条压缩至 6,373 条，保留 47%，有效过滤无关噪声，大幅降低下游分析压力。bpftrace 进程内存占用约 4MB，相对推理引擎数 GB 内存可忽略不计。eBPF 程序运行于内核空间，无需在用户态额外占用推理进程的 CPU 和内存资源。

综合来看，HybriChain 在几乎不影响推理引擎性能的前提下完成了跨语言调用链追踪。

[第 15 页 异常行为发现]

基于调用链图谱，我们实施了三类规则驱动异常检测：在 Qwen2-0.5B 模型 23.83 秒采样中，发现 30 条 HIGH 级别的 futex 阻塞告警，最大单次阻塞达 15.2 秒，并发峰值 9 条告警同时触发，揭示了 CPU-only 环境下多线程同步的结构性瓶颈与 DoS 脆弱点。此外还检测到异常大块内存分配和敏感路径访问事件，验证了调用链下游分析在多种安全场景中的适用性。

[第 16 页 创新点]

总结本文的三个创新点：

第一，首次将 eBPF 内核级动态追踪与静态二进制符号分析结合用于 AI 推理引擎，克服了纯静态方案无法获取运行时行为、纯动态方案无法还原完整调用图的各自局限。

第二，针对多语言混合架构中多种命名空间割裂的问题，提出了统一的五层语义分类模型。更进一步，设计了基于前缀特征的命名还原技术，将含哈希后缀的语言绑定函数还原为原始函数名，从而实现精确的跨语言符号分类，该技术不绑定特定引擎，适用于所有多语言混合推理框架。

第三，提出了时间戳对齐的置信度缝合算法，将微秒级和毫秒级异构事件流按时间窗对齐，自动判定 Confirmed、Inferred、Unknown 三种置信状态，解决了异构数据源无法直接拼接的难题。

[第 17 页 研究局限与未来展望]

当前局限性有四点：未覆盖 GPU 推理路径，当前实验在 CPU-only 环境下进行；召回率受探针覆盖限制，当前探针仅挂载 4 个核心函数；采样时长有限，部分长尾调用路径未被触发，属于 PoC 切片；异常检测依赖人工规则，对未知异常类型缺乏自适应能力。

未来工作方向包括：引入 CUPTI 追踪 CUDA API，建立 Python 到 CUDA kernel 完整调用链；增加 uprobe 目标函数数量，提升召回率覆盖率；引入自编码器或 LSTM 实现智能化异常检测，减少对人工规则的依赖。

[第 18 页 谢谢]

以上就是我的汇报，感谢各位老师，请批评指正。

时间分配参考

段落	预估时长
标题 + 汇报结构	0:30
研究背景	0:45
相关工作	1:00
研究目标与方法	0:45
架构 + 探针 + 降噪 + 缝合 + 符号体系	2:30
Ollama 特殊工程问题及解决方案	0:45
实验设计与结果	2:00
异常行为发现	0:45
创新点	0:45
局限展望 + 谢谢	0:45
合计	约 9:15